

Contents

1	BudgetFlow: Value-Aware Budget Allocation for LLM Agent Tasks Under a Shared Hard Cap	2
1.1	Abstract	2
1.2	1. Introduction	2
1.3	2. Related Work	4
1.3.1	2.1 LLM Cascade Routing	4
1.3.2	2.2 Per-Example Budget Enforcement	5
1.3.3	2.3 Explainable and Agent-Level Budget Routing	5
1.3.4	2.4 Positioning	6
1.4	3. BudgetFlow Framework	6
1.4.1	3.1 Task Value and Budget Model	7
1.4.2	3.2 Budget Regime Compiler	7
1.4.3	3.3 Runtime Value-Aware Policy	7
1.5	4. Experimental Setup	8
1.5.1	4.1 Task Workload	8
1.5.2	4.2 Shared Budget	8
1.5.3	4.3 Baselines	9
1.5.4	4.4 Metrics	9
1.5.5	4.5 Sensitivity Dimensions	9
1.6	5. Results	10
1.6.1	5.1 Main Result	10
1.6.2	5.2 Cost Is Not Token Price	10
1.6.3	5.3 Sensitivity Analysis	11
1.7	6. Discussion	14
1.7.1	6.1 From Cost Minimization to Value Maximization	14
1.7.2	6.2 When BudgetFlow Spends More	14
1.7.3	6.3 Relationship to Prior Work	15
1.7.4	6.4 Limitations and Future Work	15
1.8	7. Conclusion	15
1.9	References	16

1 BudgetFlow: Value-Aware Budget Allocation for LLM Agent Tasks Under a Shared Hard Cap

1.1 Abstract

We introduce BudgetFlow, a budget governance layer that maximizes normalized verified resolved value (Yield) under a single shared hard budget across a fixed batch of tasks. Existing approaches to cost-efficient LLM deployment—cascade routing, per-example budget enforcement, and budget-aware agent frameworks—operate at the level of individual queries or agent runs. None address the allocation problem that arises when a fixed task batch must share one hard budget: which tasks deserve scarce model opportunities, and how should runtime model selection reflect pre-registered task value? BudgetFlow answers this with two components: a pre-run budget regime compiler that translates a hard cap into an auditable allocation scheme, and a runtime value-aware policy that selects models at task start based on expected verified value. In experiments using a fixed 30-task SWE-bench workload under a shared budget, BudgetFlow achieves 17/30 pass, Yield 21.0, Cost 6.0000, and Yield/3.5000, compared to T2-only (12/30, Yield 14.5, Yield/\$ 2.4167) and T3-only (16/30, Yield 18.5, Yield/\$ 3.1731) baselines. The results hold directionally across KV-cache discount levels, value profiles, and budget cap settings. BudgetFlow demonstrates that shared-budget value maximization, rather than per-query cost minimization, is the appropriate objective when a batch of heterogeneous tasks competes for one hard budget.

1.2 1. Introduction

Consider a platform operator managing 30 bug reports that must be resolved by end of week. The operator has \$6.00 in API budget—not per task, but total. Some of these bugs are critical (customer-facing, revenue-impacting); others are cosmetic. The operator must decide: should the expensive, more capable model be reserved for high-value bugs, or should every task receive the same treatment? If the budget runs out before the critical bugs are reached, the operator has failed—not because the models underperformed, but because the *allocation* was wrong.

This is the shared-budget allocation problem. Large language model (LLM) agents are increasingly deployed to complete batches of tasks—sweeping issues in a repository, processing customer requests, or running scheduled maintenance workflows. In these deployments, the operator sets a single hard budget cap that the entire batch must stay under. When the cap is binding, the allocation problem is inescapable: which tasks deserve scarce model opportunities, and how should runtime model selection reflect pre-registered task value, such that total verified value is maximized under the cap?

Existing work on cost-efficient LLM deployment does not answer this question. On one hand, cascade routing systems—FrugalGPT (Chen et al., 2023), RouteLLM (Ong et al., 2024), HybridLLM (Ding et al., 2024), UCCI (Kotte, 2026), and RouteNLP (Guo et al., 2026)—optimize per-query cost-accuracy trade-offs. They decide whether to escalate a single query from a cheap model to an expensive one, but operate each query in isolation: they do not manage a shared budget across queries, track cumulative spending, or weigh task value in the escalation decision. On the other hand, per-example budget enforcement methods—INTENT (Liu et al., 2025), BATS (Liu et al., 2025), Predictive Scheduling (Brown et al., 2025), BudgetThinker (Wen et al., 2025), and BRPO (Qi et al., 2025)—enforce per-example or per-run token budgets. An agent that exceeds its individual budget is terminated. These systems enforce local constraints but make no cross-task tradeoffs. Explainable routing frameworks such as Topaz (Okamoto et al., 2026) provide auditability but treat the budget as a parameter to maximize against, not a depletable hard cap. Agent-level budget systems such as BCAS (McCleary & Ghawaly, 2026) surface remaining budget to an agent during iterative retrieval, but do not allocate across heterogeneous tasks.

In short, no existing system combines four properties: (1) one shared hard budget across a fixed batch of tasks, (2) pre-registered per-task value that is not post-hoc fitted, (3) a runtime policy that allocates scarce shared budget to maximize total verified value, and (4) a pre-run budget regime compiler that makes the cap auditable.

BudgetFlow provides these four properties. The idea is simple. A **budget regime compiler** takes a hard budget cap and a batch of tasks with pre-registered value annotations and produces an auditable allocation plan *before any tokens are spent*. A **runtime value-aware policy** then selects models at task start based on three signals: (i) the task’s pre-registered value, (ii) expected cost and expected verification probability for each eligible model, and (iii) remaining budget headroom for higher-value downstream tasks. The policy tracks cumulative spending against the cap and tightens downstream decisions as the budget depletes. The regime says what is *eligible*; the policy decides what to *use*.

BudgetFlow is not a cheapest-model fallback policy. It is a value-aware allocation policy: it may spend *more* when expected verified value justifies the spend, as long as it remains under the shared cap. In agentic repair, token price is not task-level cost—a nominally cheaper model can become more expensive when it takes more turns or stalls, while a stronger model can fail quickly and cheaply. The central comparison is therefore not who spends less, but who converts the shared budget into more verified value.

On a fixed 30-task SWE-bench workload under a binding shared budget (6.00), *BudgetFlow* achieves $\ast 17/30$ pass, *Yield* 21.0, and *Yield* / 3.5000. **Compared to uniform-tier baselines, this represents a 45% higher Yield than T2-only^{\ast\ast} (14.5) and a 13.5% improvement over T3-only (18.5)**—all within the same hard cap. More strikingly, BudgetFlow and T2-only both spend the full \$6.00, yet BudgetFlow extracts over 3.5 units of verified value per dollar versus 2.4. The directional advantage persists

across KV-cache discount levels, value profiles, and budget cap settings.

Beyond raw performance, BudgetFlow treats the budget as a governance primitive. The cap is auditable. The value annotations are pre-registered and inspectable. The regime compiler produces a plan that can be reviewed before execution. A developer can ask “why was this task given T3 while that task was given T2?” and receive an answer grounded in declared value, not in opaque model confidence scores.

To summarize, our contributions are: (1) we identify the shared-budget allocation problem—maximizing verified value under one hard cap across heterogeneous tasks—as distinct from per-query cost minimization; (2) we introduce BudgetFlow, a two-component governance layer (regime compiler + runtime value-aware policy) that solves this problem; (3) we demonstrate on a 30-task SWE-bench workload that BudgetFlow achieves the highest Yield and Yield/\$ versus uniform-tier baselines; and (4) we show through systematic sensitivity analysis that BudgetFlow’s advantage is directionally robust. This paper evaluates task-start value-aware allocation; finer-grained within-task routing, escalation, stop-loss, and learning mechanisms are left for future work.

1.3 2. Related Work

We organize related work into three clusters, each addressing one dimension of cost-aware LLM deployment. None combines all four properties of BudgetFlow: shared hard budget, pre-registered task value, verified outcomes, and auditable budget compilation.

1.3.1 2.1 LLM Cascade Routing

Cascade routing is the dominant paradigm for cost-efficient LLM inference. FrugalGPT (Chen et al., 2023) established the foundational strategy: query models from cheapest to most expensive, using a scoring model to evaluate answer quality and decide when to stop. RouteLLM (Ong et al., 2024) replaced the scoring model with a BERT-style classifier trained on human preference data. HybridLLM (Ding et al., 2024) routes based on predicted performance gaps between small and large models. AutoMix (Aggarwal et al., 2024) introduced self-verification into the cascade loop.

These systems operate per-query. Each query is an independent cost-accuracy tradeoff. They do not allocate a shared budget across a batch, track cumulative spending, or prioritize high-value tasks. RouteNLP (Guo et al., 2026) moves closer to deployment-level routing with a closed-loop conformal cascading framework, but its budget-based mode uses dynamic programming to maximize quality under a per-workflow cost cap, not a shared hard budget that depletes across an ordered task sequence.

UCCI (Kotte, 2026, arXiv:2605.18796) addresses a specific weakness in cascade routers—uncalibrated confidence scores—by mapping token-level margin uncertainty to per-query error probabilities via isotonic regression. On a production NER workload, UCCI reduced inference cost by 31% at micro-F1 0.91. However, it remains a per-query cascade and does not manage shared-budget allocation.

1.3.2 2.2 Per-Example Budget Enforcement

A parallel literature enforces compute budgets within a single model call, reasoning trace, or agent run. Predictive Scheduling (Brown et al., ICML 2025) uses lightweight MLP probes to predict optimal reasoning length before generation begins. e1 (Kleinman et al., 2025) uses RL to train models that follow user-specified effort fractions. BRPO (Qi et al., ICML 2025) trains models for optimal performance at any thinking budget via truncated chain-of-thought traces. BudgetThinker (Wen et al., 2025) inserts control tokens to signal remaining token budget during generation. The BAR Conjecture (Zhou et al., 2025) provides formal grounding by proving that no LLM system can simultaneously optimize inference-time budget, factual authenticity, and reasoning capacity beyond a critical input size.

On the tool-call side, INTENT (Liu et al., ICML 2025) formalizes budget-constrained tool use with priced, stochastic calls and Monte Carlo lookahead. BATS (Liu et al., 2025) identifies a failure mode—giving agents larger tool-call budgets is ineffective without budget awareness—and introduces a lightweight Budget Tracker plugin. AVA (Patel et al., TMLR 2026) combines adaptive search, uncertainty estimation, and verification cascades under explicit budgets. Fang et al. (2026) extend per-example budget control to multi-hop QA search agents with a Value-of-Information controller under dual token and tool-call budgets.

These systems enforce per-example or per-run constraints. A run that exceeds its budget is terminated. They do not address the cross-task allocation question: how should a batch of tasks of varying value share one hard budget?

1.3.3 2.3 Explainable and Agent-Level Budget Routing

Topaz (Okamoto et al., 2026, CHI 2026 Workshop on Human-Centered Explainable AI, arXiv:2604.03527) introduces explainable model routing using an 8-dimensional skill taxonomy. It profiles both models and subtasks, provides objective-based and budget-based routing modes, and records all decisions in structured trace logs. Topaz’s contribution is auditability—a developer can ask whether the system was smart or just cheap. However, its budget is a parameter to maximize against, not a binding constraint that depletes in real time.

BCAS (McCleary & Ghawaly, 2026) provides a model-agnostic evaluation harness that surfaces remaining budget to an agent during iterative RAG. It gates tool calls against explicit token and turn budgets.

A key finding—accuracy improves with additional searches only up to a small cap—directly informs when a budget-constrained agent should stop spending on a given task. AgentServe (Zhang et al., 2026) addresses GPU-level resource contention in multi-agent serving but does not handle policy-level allocation across heterogeneous tasks.

1.3.4 2.4 Positioning

Approach	Decision Unit	Budget Scope	Value Awareness	Verified Outcomes
FrugalGPT / RouteLLM / HybridLLM	Per-query	Cost minimization	No	No
UCCI	Per-query	Cost minimization	No	No
RouteNLP	Per-workflow	Per-workflow cap	Quality sensitivity	No
Predictive Scheduling / e1 / BRPO / BudgetThinker	Per-trace	Per-example budget	No	No
INTENT / BATS / AVA	Per-agent-run	Per-run budget	Binary	Task success
Topaz	Per-subtask	Cost-sensitivity parameter	Indirect	No
BCAS	Per-agent-session	Token/turn budget	No	No
BudgetFlow	Per-task in fixed batch	Shared hard budget	Pre-registered value	Verified resolution

1.4 3. BudgetFlow Framework

BudgetFlow consists of two components: a pre-run budget regime compiler and a runtime value-aware allocation policy. Together they govern how a fixed batch of tasks, each with a pre-registered

value, consumes one shared hard budget.

1.4.1 3.1 Task Value and Budget Model

Let a batch of n tasks $T = \{t_1, \dots, t_n\}$ be submitted for execution. Each task t_i carries a pre-registered value $v_i > 0$, which reflects the operator’s judgment of the task’s importance. This value is declared before execution begins—it is not estimated, inferred, or fitted post-hoc.

The operator sets a hard budget B , a single cap on total token-level expenditure across all tasks in the batch. Once the batch starts executing, total spending must not exceed B . If the budget is exhausted before all tasks are processed, remaining tasks are not executed. The objective is to maximize total verified resolved value (Yield) across the batch, where a task contributes its pre-registered value v_i if and only if its output passes verification:

$$\text{Yield} = \sum_{i: \text{task } t_i \text{ passes verification}} v_i$$

The operator’s problem is therefore: given T , $\{v_i\}$, and B , select a model assignment policy that maximizes expected Yield.

1.4.2 3.2 Budget Regime Compiler

The budget regime compiler takes the budget cap B , the task batch T , and the value annotations $\{v_i\}$ and produces a regime—a structured plan that partitions the budget into allocation tiers and specifies, for each tier, which model options are available and under what conditions they may be selected.

The compiler does not execute tasks. It produces an auditable artifact that can be inspected before any tokens are spent. The regime is static for the duration of the batch, but the runtime policy retains discretion within its constraints: a regime says which models are *eligible* at each value tier; the runtime policy decides which eligible model to *use* for each specific task, informed by expected cost and expected verification outcome.

1.4.3 3.3 Runtime Value-Aware Policy

At the start of each task t_i , the runtime policy reads the task’s pre-registered value v_i , the current remaining budget B_{rem} , and the budget regime. It selects a model $m \in \mathcal{M}$ that is eligible under the regime for tasks of value v_i , considering:

1. The expected cost of executing t_i with model m , given historical cost data for similar tasks.

2. The expected probability that t_i will pass verification if executed with model m .
3. The remaining budget headroom: a model that is otherwise optimal may be skipped if selecting it would leave insufficient budget for higher-value downstream tasks.

The policy may select a stronger model when the expected verified value gain justifies the additional expected cost, and may select a cheaper model when the value at stake does not warrant the spend. The policy tracks cumulative spending and updates B_{rem} after each completed task, allowing downstream decisions to tighten as the budget depletes.

Critically, model selection occurs at task start and is governed by the shared budget constraint. The policy does not perform per-turn escalation, segment-level routing, or stop-loss within a task—those mechanism-level behaviors are reserved for future investigation (Claim 2).

1.5 4. Experimental Setup

We evaluate a single question: under a fixed shared hard budget, does BudgetFlow’s value-aware allocation achieve higher Yield and higher Yield per Dollar than uniform-tier baselines?

1.5.1 4.1 Task Workload

We use a fixed batch of 30 tasks drawn from the SWE-bench verification suite. Each task is a real-world software engineering issue requiring an agent to produce a patch and pass verification. The 30-task batch is fixed across all runs; no task is added, removed, or reordered between conditions.

Each task carries a pre-registered value. We evaluate under three value profiles: (1) equal value—all tasks have identical value, isolating the allocation effect from value heterogeneity; (2) current value—tasks are annotated with the value scheme used in our production budget regime; (3) wide gradient—task values span a larger range to stress-test the policy’s ability to prioritize under steep value differences. Unless otherwise noted, results are reported under the current value profile.

1.5.2 4.2 Shared Budget

A single hard budget cap B is set before the batch begins. Once set, the cap is binding: total spending across all 30 tasks must not exceed B . The cap is chosen to be active: T2-only and BudgetFlow reach the full \$6.0000 cap, while T3-only finishes at \$5.8303. BudgetFlow’s regime compiler receives the same cap and must allocate within it.

1.5.3 4.3 Baselines

We compare three policies:

- **T2-only:** Every task is executed with model T2. Spending is typically below the cap; unused budget is wasted.
- **T3-only:** Every task is executed with model T3. Spending may approach or reach the cap, but every task receives the same model regardless of value.
- **BudgetFlow:** The budget regime compiler produces a regime for the given cap and value profile. The runtime policy selects T2 or T3 at task start based on task value, expected cost, expected verification probability, and remaining budget.

All policies operate under identical task order, identical verification criteria, and the same hard budget cap.

1.5.4 4.4 Metrics

- **Pass count:** Number of tasks (out of 30) whose agent-produced patch passes verification.
- **Yield:** Sum of pre-registered values of all passing tasks. This is the primary metric—it directly measures verified value produced under the budget.
- **Yield/\$:** Yield divided by total cost. This measures budget efficiency—how much verified value each dollar of the shared budget produces.
- **Total Cost:** Aggregate token expenditure across all 30 tasks, at or below the cap.

1.5.5 4.5 Sensitivity Dimensions

We evaluate BudgetFlow’s robustness across three sensitivity dimensions:

- **KV-cache discount:** Different KV-cache pricing levels (KV0, KV50, KV80, KV90, KV98, KV99) affect the token-level cost accounting of each model. We report Yield and Yield/\$ across the tested levels.
- **Value profile:** Equal-value, current-value, and wide-gradient value annotations test whether BudgetFlow’s advantage depends on a specific value distribution.
- **Budget cap:** Tighter and more binding caps test whether BudgetFlow maintains its directional advantage as budget pressure increases.

1.6 5. Results

1.6.1 5.1 Main Result

Table 1 reports the primary comparison under the current value profile and the active budget cap.

Policy	Pass (out of 30)	Yield	Total Cost	Yield/\$
T2-only	12	14.5	\$6.0000	2.4167
T3-only	16	18.5	\$5.8303	3.1731
BudgetFlow	17	21.0	\$6.0000	3.5000

Table 1. Main result on the 30-task SWE-bench workload under a shared hard budget cap. BudgetFlow achieves the highest pass count, highest Yield, and highest Yield per Dollar.

BudgetFlow achieves 17/30 pass, compared to 16/30 (T3-only) and 12/30 (T2-only). Yield reaches 21.0, exceeding T3-only (18.5) by 13.5% and T2-only (14.5) by 44.8%. BudgetFlow and T2-only both record total cost of 6.0000 at the cap, but BudgetFlow extracts substantially more verified value from the same budget. Yield for BudgetFlow is 3.5000, compared to 3.1731 (T3-only) and 2.4167 (T2-only).

These results illustrate a central dynamic of shared-budget allocation. T2-only and BudgetFlow both spend the full cap (6.0000), but BudgetFlow extracts substantially more verified value from the same budget—Yield 3.5000 vs. 2.4167. T3-only cannot differentiate: it grants the same model to every task, missing the opportunity to conserve budget on low-value tasks and concentrate spend on high-value ones. Its lower total cost (\$5.8303) does not translate into higher efficiency.

1.6.2 5.2 Cost Is Not Token Price

The T3-only baseline records a lower total cost (\$5.8303) than BudgetFlow (\$6.0000). A naive reading would favor T3-only for being cheaper. This reading is incorrect.

BudgetFlow is not a cheapest-model fallback policy. It is a value-aware budget allocation policy: it may spend more when expected verified value justifies the spend, as long as it remains under the shared cap. In agentic repair, token price is not task-level cost; a nominally cheaper model can become more expensive when it takes more turns or stalls, while a stronger model can fail quickly and cheaply. Therefore, the central comparison is whether a policy converts the shared budget into more normalized verified value.

BudgetFlow spends the full cap and produces more verified value. T3-only is a strong efficiency baseline with higher Yield/\$ than T2-only; however, its lower spend does not dominate when BudgetFlow

achieves both higher Yield (21.0 vs. 18.5) and higher Yield/\$ (3.5000 vs. 3.1731) within the same shared cap. The operator’s objective is to maximize verified value under the cap. On that objective, BudgetFlow is the best policy on the paper’s primary metric, Yield, and also improves Yield/\$ in this read-out.

1.6.3 5.3 Sensitivity Analysis

We evaluate BudgetFlow’s robustness across three sensitivity dimensions: KV-cache discount, value profile, and budget cap. All sensitivity runs use the same 30-task batch, fixed task order, and frozen CostSource. The sensitivity readout uses corrected Yield values from the patch-cleaner forensic re-read (BudgetFlow Yield 22.0 vs. Table 1’s uncorrected 21.0); the directional pattern is unchanged. All three baselines consume the same task set under the same protocol.

KV-cache discount sensitivity. Table 2 reports Yield and Yield/\$ across six KV-cache discount levels under the current value profile. KV-cache discounting reduces the effective per-token cost of repeated input tokens without changing base model prices or physical model bindings; higher discount levels benefit models that produce longer multi-turn trajectories with large repeating prefixes.

KV Discount	T2 Cost / Y\$	T3 Cost / Y\$	BF Cost / Y\$	Yield Winner	Efficiency Winner
0%	\$17.9595 / 1.0858	\$8.7395 / 2.0024	\$12.7868 / 1.7205	BF 22.0	T3
50%	\$9.7729 / 1.9953	\$5.1336 / 3.4089	\$7.1727 / 3.0672	BF 22.0	T3
80%	\$4.8610 / 4.0116	\$2.9701 / 5.8921	\$3.8043 / 5.7830	BF 22.0	T3
90%	\$3.2236 / 6.0491	\$2.2489 / 7.7816	\$2.6814 / 8.2045	BF 22.0	BF
98%	\$1.9138 / 10.1893	\$1.6720 / 10.4667	\$1.7832 / 12.3374	BF 22.0	BF
99%	\$1.7500 / 11.1426	\$1.5998 / 10.9385	\$1.6709 / 13.1664	BF 22.0	BF

Table 2. KV-cache discount sensitivity. BudgetFlow maintains the highest Yield (22.0) at every discount level. Pure T3 leads Yield/\$ at lower discount levels (KV0–KV80), where its lower token con-

sumption dominates; BudgetFlow overtakes on Yield/\$ at KV90 and above, where KV-cache discounts compress the cost gap between T2 and T3 trajectories.

BudgetFlow holds the highest absolute Yield at every discount level. The efficiency ranking depends on the discount regime: at low-to-moderate KV discounts (0%–80%), pure T3’s lower total token consumption yields the best Yield/. *At KV90 and above, BudgetFlow overtakes pure T3 on Yield/*, because the cost penalty for BudgetFlow’s T2-routed tasks shrinks under aggressive KV-cache pricing. This confirms that KV-cache assumptions affect efficiency rankings but not the Yield ordering, and supports treating KV-cache as an explicit sensitivity rather than a hidden CostSource parameter.

Value profile sensitivity. Table 3 reports Yield for all three policies across nine pre-registered value profiles at KV50. Value profiles vary the mapping from task criticality to numerical weight, testing whether BudgetFlow’s advantage is an artifact of a specific value scheme.

Value Profile	T2 Yield	T3 Yield	BF Yield	BF vs Best Baseline
equal	18.0	16.0	20.0	+2.0
current	19.5	17.5	22.0	+2.5
current_high_to_2.0	21.0	19.0	24.0	+3.0
current_high_to_2.5	22.5	20.5	26.0	+3.5
top20_effort_critical	22.5	20.5	25.0	+2.5
top33_effort_critical	24.5	21.5	28.5	+4.0
effort_tertiles_1_1.5_2.5	27.5	24.0	31.5	+4.0
both_fail_critical	19.5	17.5	23.5	+4.0
top10_effort_critical	24.5	21.5	28.5	+4.0

Table 3. Value profile sensitivity at KV50. BudgetFlow achieves higher Yield than both baselines across all nine profiles. The margin grows with value spread: BudgetFlow’s advantage is smallest under equal value (+2.0 over T2’s 18.0) and largest under the widest gradient profiles (effort_tertiles, +4.0 over T2’s 27.5).

BudgetFlow outperforms both baselines on Yield under every tested value profile. The equal-value profile—where every task has identical weight—is the hardest test for value-aware allocation, since it removes the signal the policy uses to prioritize. BudgetFlow still leads (+2.0), indicating that the regime compiler’s cost-aware default allocation provides value even when value differences are absent. As the value spread widens, BudgetFlow’s advantage grows: under effort_tertiles_1_1.5_2.5, the widest gradient tested, BudgetFlow leads the best baseline by +4.0. This confirms that value-aware

allocation captures more of the available gain when task values are heterogeneous—precisely the setting that motivates a shared-budget governance layer.

Budget cap sensitivity. Table 4 reports Yield, total cost, and Yield/\$ across eight budget cap levels at KV50 under the current value profile. The cap is a hard shared constraint; when binding, later tasks in the sequence are not executed.

Cap	T2 Yield / Cost / Y\$	T3 Yield / Cost / Y\$	BF Yield / Cost / Y\$	Yield Winner	Efficiency Winner
\$3.00	9.5 / \$2.9401 / 3.2312	10.0 / \$2.9445 / 3.3961	13.5 / \$2.9813 / 4.5283	BF	BF
\$4.00	12.5 / \$3.9541 / 3.1612	14.5 / \$3.9682 / 3.6540	16.0 / \$3.9972 / 4.0028	BF	BF
\$5.00	12.5 / \$4.9650 / 2.5176	17.5 / \$4.9101 / 3.5641	18.0 / \$4.9581 / 3.6304	BF	BF
\$6.00	13.5 / \$5.9444 / 2.2710	17.5 / \$5.1336 / 3.4089	20.0 / \$5.9992 / 3.3338	BF	T3
\$7.00	15.5 / \$6.9965 / 2.2154	17.5 / \$5.1336 / 3.4089	22.0 / \$6.9740 / 3.1546	BF	T3
\$8.00	17.5 / \$7.9361 / 2.2051	17.5 / \$5.1336 / 3.4089	22.0 / \$7.1727 / 3.0672	BF	T3
\$9.00	17.5 / \$8.9609 / 1.9529	17.5 / \$5.1336 / 3.4089	22.0 / \$7.1727 / 3.0672	BF	T3
\$11.02	19.5 / \$9.7729 / 1.9953	17.5 / \$5.1336 / 3.4089	22.0 / \$7.1727 / 3.0672	BF	T3

Table 4. Budget cap sensitivity at KV50. At tight caps (\$3–5), *BudgetFlow* wins both *Yield* and *Yield/\$*. At looser caps (6+), *BudgetFlow* leads *Yield* but pure T3 leads *Yield/\$* because T3 saturates at \$5.13—it solves the T3-solvable subset and cannot convert additional budget into more verified value.

The cap sensitivity reveals two regimes. At binding caps (\$3–5), *where every policy is under genuine budget pressure*, *value-aware* allocation is both more productive and more efficient. At looser caps (\$6+), pure T3 saturates: its cost flattens at \$5.1336 and its Yield plateaus at 17.5, because the remaining unsolved tasks require capabilities beyond T3. *BudgetFlow* continues to convert additional budget into higher Yield (reaching 22.0 at 7), *but its Yield/\$* falls below pure T3 because the extra tasks it solves require T3-level spend on T2-viable tasks. This is not a policy failure—it is the expected behavior when a

value-aware policy spends more to capture value that a pure-tier baseline cannot reach. T2-only scales Yield gradually with cap but never catches BudgetFlow on absolute Yield.

The key takeaway from the cap sensitivity is that BudgetFlow’s mechanism is strongest when the cap is genuinely binding—exactly the regime the budget regime compiler targets. When the cap is loose enough that pure T3 is unconstrained, the allocation problem is less interesting; the operator should tighten the cap or enlarge the workload.

1.7 6. Discussion

1.7.1 6.1 From Cost Minimization to Value Maximization

The dominant mindset in cost-efficient LLM deployment is cost minimization: route queries to the cheapest model that can handle them, escalate only when necessary, and treat every saved token as a win. This mindset is appropriate when each query is an independent unit with no shared resource constraint and no differential value.

BudgetFlow addresses a different setting. When an operator sets one hard budget for a batch of tasks, the objective is not to minimize the cost of each task but to maximize the total value the batch produces before the budget runs out. In this setting, spending more on a task is not a failure—it is the correct decision when the task’s verified value exceeds the value that the same budget would produce if allocated elsewhere.

This is a governance choice, not merely an engineering optimization. The budget cap is an auditable constraint set by the operator. The value annotations are pre-registered and inspectable. The regime compiler produces a plan that can be reviewed before execution. Together, these properties make the allocation decision transparent—a developer can ask “why was this task given T3 while that task was given T2?” and receive an answer grounded in value, not in opaque model confidence scores.

1.7.2 6.2 When BudgetFlow Spends More

In the current results, BudgetFlow and T2-only both spend the full cap (\$6.0000), but BudgetFlow achieves substantially higher Yield (21.0 vs. 14.5). T3-only spends \$5.8303. The 0.1697 difference between BudgetFlow and T3—only is not a policy failure—T3—only is a strong efficiency baseline than T2-only. However, the fact that lower spend does not translate into higher verified value reinforces the central point: cost minimization is not the right objective under a shared budget cap.

BudgetFlow’s runtime policy may select T3 for high-value tasks even when T2 would be cheaper in expectation, because the expected gain in verified value outweighs the expected additional cost. Conversely, it may select T2 for low-value tasks even when budget remains, because the value at stake does not justify the spend. This is the allocation decision, and it is what produces the higher Yield and Yield/\$ in Table 1.

1.7.3 6.3 Relationship to Prior Work

BudgetFlow’s architecture is compatible with several existing techniques. UCCI’s calibrated uncertainty estimates could inform the runtime policy’s expected verification probability model. Topaz’s explainable skill taxonomy could enrich the value registration schema. The budget regime compiler’s audit trail aligns with Topaz’s goal of making routing decisions inspectable. These integrations are natural extensions and do not require architectural changes to BudgetFlow’s two-component structure.

1.7.4 6.4 Limitations and Future Work

The current evaluation uses a fixed 30-task workload from SWE-bench. Results may not generalize to workloads with different task distributions, different verification criteria, or different model families. The value annotations are human-assigned; automated value inference from task metadata is an open direction. This paper evaluates task-start value-aware allocation; finer-grained within-task routing, escalation, stop-loss, and learning mechanisms are left for future work.

1.8 7. Conclusion

BudgetFlow demonstrates that under a shared hard budget, value-aware allocation across a fixed task batch achieves higher normalized verified resolved value (Yield) and higher Yield per Dollar than uniform-tier baselines. On a 30-task SWE-bench workload, BudgetFlow achieves 17/30 pass, Yield 21.0, and Yield/\$ 3.5000, exceeding T3-only (16/30, Yield 18.5, Yield/\$ 3.1731) and T2-only (12/30, Yield 14.5, Yield/\$ 2.4167). The directional advantage persists across KV-cache discount levels, value profiles, and budget cap sensitivities.

The core insight is that shared-budget allocation is a value maximization problem, not a cost minimization problem. A nominally stronger model is not “too expensive” if it converts budget into verified value more effectively. BudgetFlow treats the budget as a governance constraint—auditable, pre-

declared, and active—and the allocation policy as the mechanism for converting that constraint into the most verified value possible.

1.9 References

- [1] Chen, L., Zaharia, M., & Zou, J. (2023). FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv:2305.05176*.
- [2] Ong, I., et al. (2024). RouteLLM: Learning to Route LLMs with Preference Data. *arXiv:2406.18665*.
- [3] Ding, S., et al. (2024). HybridLLM: Dynamic Model Selection for Cost-Efficient LLM Applications.
- [4] Aggarwal, P., et al. (2024). AutoMix: Cascading with Self-Verification Results.
- [5] Guo, D., Wu, J., & Yiu, S. (2026). RouteNLP: Closed-Loop LLM Routing with Conformal Cascading and Distillation Co-Optimization.
- [6] Kotte, V. (2026). UCCI: Calibrated Uncertainty for Cost-Optimal LLM Cascade Routing. *arXiv:2605.18796*.
- [7] Okamoto, M., Erol, A. K., & Riedl, M. (2026). Explainable Model Routing for Agentic Workflows (Topaz). CHI 2026 Workshop on Human-Centered Explainable AI. *arXiv:2604.03527*.
- [8] Brown, D., Muppidi, S., & Shahout, R. (2025). Predictive Scheduling for Efficient Inference-Time Reasoning in LLMs. ICML 2025.
- [9] Kleinman, M., et al. (2025). e1: Learning Adaptive Control of Reasoning Effort.
- [10] Qi, W., et al. (2025). Optimizing Anytime Reasoning via Budget Relative Policy Optimization. ICML 2025.
- [11] Wen, Z., et al. (2025). BudgetThinker: Empowering Budget-Aware LLM Reasoning with Control Tokens. *arXiv:2508.17196*.
- [12] Zhou, X., et al. (2025). BAR Conjecture: the Feasibility of Inference Budget-Constrained LLM Services. *arXiv:2507.23170*.
- [13] Liu, C., et al. (2025). Budget-Constrained Agentic LLMs: Intention-Based Planning for Costly Tool Use (INTENT). ICML 2025.
- [14] Liu, C., et al. (2025). Budget-Aware Tool-Use Enables Effective Agent Scaling (BATS). *arXiv:2511.17006*.
- [15] Patel, K., et al. (2026). Anytime Verified Agents (AVA). TMLR 2026.

- [16] McCleary, K. & Ghawaly, J.M. (2026). Quantifying the Accuracy and Cost Impact of Design Decisions in Budget-Constrained Agentic LLM Search (BCAS).
- [17] Zhang, Y., et al. (2026). AgentServe: Algorithm-System Co-Design for Efficient Agentic AI Serving.
- [18] Moslem, Y. & Kelleher, J.D. (2026). Dynamic Model Routing and Cascading for Efficient LLM Inference: A Survey.
- [19] Fang, Z., Hu, S., Chang, Z., Guo, Y., Tao, Y., Liu, H., Ruan, M., Huang, J., & Fang, Y. (2026). Inference-Time Budget Control for LLM Search Agents.